EXP.NO:	4	SYSTEM CALLS: fork(), exec(), getpid(), opendir(), readdir()
DATE:		

AIM

To study and implement Linux system calls — fork(), exec(), getpid(), opendir(), and readdir() using C programming.

PROCEDURE

1. Write a C program using the appropriate system call.
2. Compile using: gcc filename.c
3. Execute using: ./a.out
4. Observe and record the output.

PROGRAM 1 – fork() and getpid()

```
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        printf("Fork Failed\n");
    }
    else if (pid == 0) {
        printf("Child Process - PID: %d\n", getpid());
        printf("Child: Parent PID is %d\n", getppid());
    }
    else {
        printf("Parent Process - PID: %d\n", getpid());
        printf("Parent: Created Child PID = %d\n", pid);
    }
    return 0;
}
```

PROGRAM 2 – exec() System Call

```
#include <stdio.h>
#include <unistd.h>

int main() {
    printf("Before exec() - PID: %d\n", getpid());
    char *args[] = {"ls", "-l", NULL};
    execvp("ls", args);
    printf("This line will not execute\n");
    return 0;
}
```

PROGRAM 3 – opendir() and readdir()

```
#include <stdio.h>
#include <dirent.h>

int main() {
    DIR *dir;
    struct dirent *entry;

    dir = opendir(".");
    if (dir == NULL) {
        printf("Cannot open directory\n");
        return 1;
    }
    printf("Contents of current directory:\n");
    while ((entry = readdir(dir)) != NULL) {
        printf("%s\n", entry->d_name);
    }
    closedir(dir);
    return 0;
}
```

OUTPUT

```
--- Program 1: fork() and getpid() ---
$ gcc fork_demo.c
$ ./a.out
Parent Process - PID: 1023
Parent: Created Child PID = 1024
Child Process - PID: 1024
Child: Parent PID is 1023

--- Program 2: exec() ---
$ gcc exec_demo.c
$ ./a.out
Before exec() - PID: 1025
total 32
-rwxr-xr-x 1 user user 8192 Apr 10 10:00 a.out
-rw-r--r-- 1 user user  210 Apr 10 09:58 exec_demo.c
-rw-r--r-- 1 user user  198 Apr 10 09:55 fork_demo.c

--- Program 3: opendir() and readdir() ---
$ gcc dir_demo.c
$ ./a.out
Contents of current directory:
.
..
a.out
dir_demo.c
exec_demo.c
fork_demo.c
employee.txt
```

RESULT

System calls fork(), exec(), getpid(), opendir(), and readdir() were implemented and executed successfully in C. The parent-child relationship, process IDs, and directory reading were observed correctly.